

FACTORY METHOD

problema

- TODA A APLICAÇÃO FOI PENSADA EM CARROS TALVEZ ATÉ COM UMA VARIÇÃO COMO O UBER BLACK, COMFORT, PRIORIDADE, PORÉM AGORA VOCÊ TEM UMA DEMANDA DA ÁREA DE NEGÓCIOS, O UBER EATS, EM VEZ DE APENAS CARROS, AGORA TEMOS MOTOS E EM GRANDES CIDADES TEMOS BISCICLETAS TAMBÉM.
- SE A MAIOR PARTE DA APLICAÇÃO ESTÁ AMARRADA A UMA CLASSE OU DE ALGUMA FORMA ESTÁ ENGESSADA, FICA DIFÍCIL ESTENDER OS RECURSOS PARA APLICAR A OUTROS MEIOS DE TRANSPORTE.

Resultado: Código sujo, if e demais condicionais com efeitos colaterais e difícil manutenibilidade.

solução

- Temos o problema agora vamos apresentar a solução...
- No Factory Method nós delegamos a instância para SUBCLASSES;
- Iremos programar dirigido a interfaces de maneira se eu precisar aumentar o recurso do meu Factory, ele tem a possibilidade de estender sem alterar o código.



Neste padrão, você deve substituir as chamadas diretas (new) de construção de objetos da classe, para um método intermediário que fará o papel de **fábrica**, este **método fábrica ou Factory Method**, irá controlar como criar os objetos que estendem a sua classe serão instanciados.

Os objetos retornados pelo método fábrica são chamados de **produtos** ou **Classes Concretas**.

Assim temos em nosso cenário a classe Veículo que implementa a interface Transporte, e as subclasses concretas carros, motos, bicicletas que herdam de Veículo e implementar os métodos declarados na interface.

A limitação fica por conta das subclasses. Caso estas retornem valores diferentes, precisam compartilhar a mesma classe ou interface **base**. O **método fábrica ou Factory Method** na **classe base** deve ter a Interface como o seu tipo de retorno.



DIAGRAMAS

